

Übung 6: Graphen I und II

(Dauer: 90 Minuten - Gruppenarbeit empfohlen)

Ziel der Übung: Sie lernen, ...

1. Betrachten Sie den folgenden ungerichteten, ungewichteten Graphen:

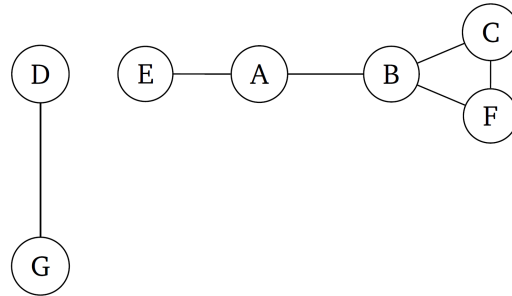


Abbildung 1: ungerichteter, ungewichteter Graph

- Bestimmen Sie $V, E, |V|, |E|$
 - Wie groß ist der maximale Grad des Graphen?
 - Enthält der Graph Zyklen? Falls ja, welche?
 - Wie sieht der einfache Pfad mit der maximalen Länge aus?
 - Was ist eine Kante, die man dem Graphen hinzufügen könnte, um die Länge des maximal langen einfachen Pfades des neuen Graphen auf 6 zu erhöhen?
 - Was sind die Zusammenhangskomponenten des Graphen?
2. Betrachten Sie den folgenden ungerichteten, gewichteten Graphen:

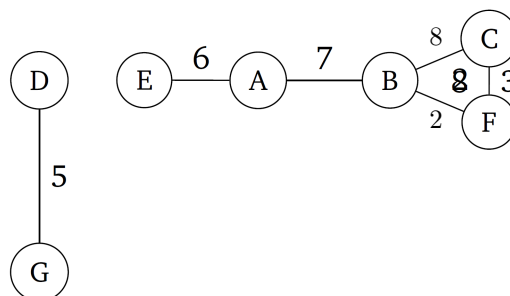


Abbildung 2: ungerichteter, gewichteter Graph

- Welches ist der Pfad mit der geringsten Anzahl von Knoten von E nach C? Wie hoch sind seine Kosten?
- Welches ist der Pfad mit den geringsten Kosten von E nach C? Wie hoch sind seine Kosten?
- Welches ist der Pfad mit der geringsten Länge von E nach C? Wie lang ist er?

3. Betrachten Sie den folgenden Graphen. Nehmen wir an, wir wollen ihn traversieren, beginnend bei Knoten A.

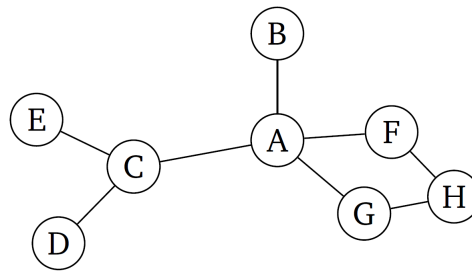


Abbildung 3: ungerichteter, ungewichteter Graph

Was sind zwei mögliche Reihenfolgen der besuchten Knoten, wenn wir sie mit Hilfe der Breitensuche durchlaufen? Was ist, wenn wir die Tiefensuche verwenden? Für die erste Reihenfolge müssen Sie den Graphen traversieren, indem Sie Dinge aus der Warteschlange/dem Stapel hinzufügen/entfernen. Schreiben Sie dafür explizit den Inhalt von `perimeter` in jeder Iteration der äußeren `while` Schleife der beiden Algorithmen auf (s. Pseudocode von DFS und BFS). Um die zweite Traversierungs-Reihenfolge für jeden Algorithmus zu ermitteln, können Sie sich den Graphen einfach ansehen.

4. Betrachten Sie den folgenden Graphen:

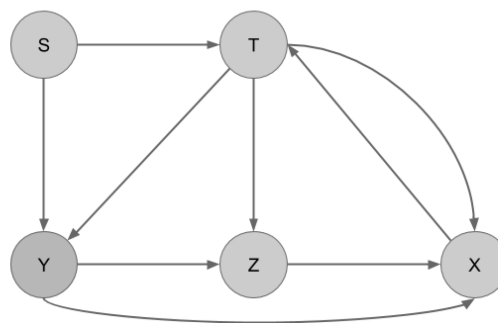


Abbildung 4: gerichteter, ungewichteter Graph

Führen Sie den Algorithmus der Breitensuche aus, um die kürzesten Pfade in diesem Graphen zu finden, ausgehend von Knoten *S*. Notieren Sie die Warteschlange der Knoten über die Iterationen und verwenden Sie die unten stehende Tabelle, um die einzelnen Schritte des Algorithmus zu verfolgen. Zeichnen Sie schließlich den resultierenden Baum kürzester Pfade, nachdem der Algorithmus beendet ist. Hinweis: Wenn zwei Knoten gleichzeitig in die Warteschlange eintreten, lösen Sie die Gleichheit auf, so dass der Knoten, der alphabetisch zuerst kommt, die Warteschlange zuerst verlässt.

Knoten	Vorgängerknoten	In <code>visited</code> gespeichert?
S		
T		
X		
Y		
Z		

5. Betrachten Sie den folgenden Graphen:

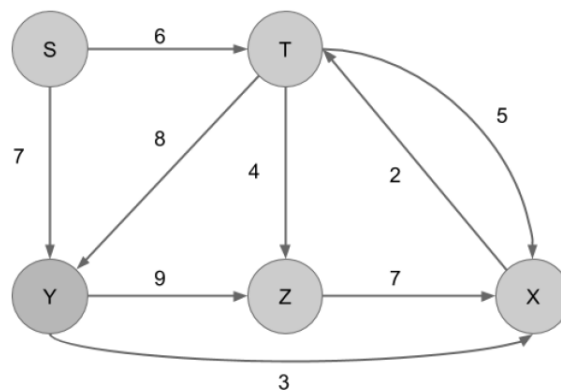
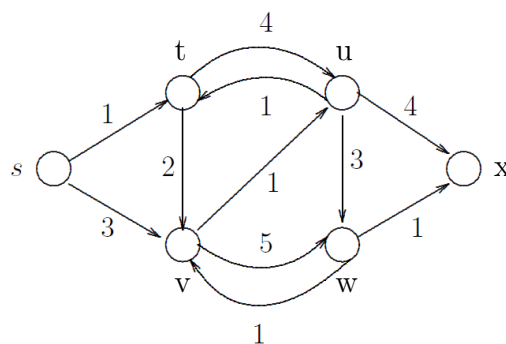


Abbildung 5: gerichteter, gewichteter Graph

Führen Sie den Dijkstra-Algorithmus auf diesem Graphen aus, beginnend mit dem Knoten *S*. Verwenden Sie die unten stehende Tabelle, um die einzelnen Schritte des Algorithmus zu verfolgen. Zeichnen Sie auch den sich ergebenden Baum kürzester Pfade, nachdem der Algorithmus beendet wurde.

Knoten	Distanz zu Startknoten	Vorgängerknoten	Verarbeitet?
S			
T			
X			
Y			
Z			

6. Kürzeste Pfade in einem gewichteten Graphen.



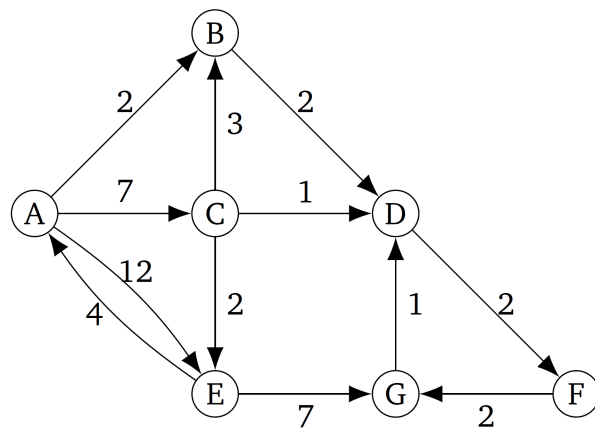
- (a) Bestimmen Sie den kürzesten Pfad vom Startknoten *s* aus zu allen anderen Knoten des obigen Graphen. Sie können dazu folgende Tabelle nutzen. Die kürzesten Abstände vom Knoten *s* aus sollen am Ende in der Spalte Distanz stehen.

Knoten	Distanz	Vorgänger	Verarbeitet?
s			
t			
u			
v			
w			
x			

- (b) Zeichnen Sie den Baum kürzester Pfade von Startknoten *s* aus.

7. Angenommen, wir versuchen, ein Labyrinth in einem 2D-Top-Down-Videospiel zu entwerfen. Die Welt wird als Gitter dargestellt, in dem jedes Feld entweder eine unpassierbare Wand, ein freies Feld, auf das der Spieler gehen kann, oder ein Wurmloch ist. In jedem Zug kann sich der Spieler um ein Feld auf dem Gitter auf ein beliebiges angrenzendes freies Feld bewegen. Steht der Spieler auf einem Wurmloch, kann er seinen Zug stattdessen dazu nutzen, sich an das andere Ende des Wurmlochs zu teleportieren, das sich irgendwo auf dem Spielfeld befindet. Nehmen wir an, es gibt mehrere Münzen, die auf der Karte verstreut sind. Ihr Ziel ist es, einen Algorithmus zu entwickeln, der einen Pfad zwischen dem Spieler und einer Münze in möglichst wenigen Zügen findet. Beschreiben Sie, wie Sie dieses Szenario als Graph darstellen würden (was sind die Knoten und Kanten? Ist dies ein gewichteter oder ungewichteter Graph? Gerichtet oder ungerichtet?). Beschreiben Sie dann, wie Sie einen Algorithmus implementieren würden, um diese Aufgabe zu lösen.
8. Sie haben gerade Ihren ersten großen DJ-Job als “DJ Kistra” ergattert. Während Ihrer Show spielen Sie “Shake It Off” und beschließen, das Tempo mit “Wildest Dreams” zu drosseln. Aber Sie wissen, dass das Publikum sehr enttäuscht sein wird, wenn Sie zwei Songs spielen, deren Tempo sich um mehr als 10 Beats pro Minute unterscheidet, oder wenn Sie nur einen Teil eines Songs spielen. Stattdessen müssen Sie eine Liste von Liedern finden, die Sie spielen können, um allmählich zu “Wildest Dreams” zu gelangen. Ihr Ziel ist es, mit einer Wiedergabeliste aus immer langsameren Liedern so schnell wie möglich (in Sekunden) zu “Wildest Dreams” überzugehen. Sie haben eine Liste mit allen Songs, die Sie spielen können, deren Geschwindigkeit in Beats pro Minute und die Länge der Songs in Sekunden.
- (a) Beschreiben Sie einen Graphen, den Sie konstruieren könnten, um das Problem zu lösen. Sie sollten mindestens angeben, welche Knoten und Kanten es gibt und ob die Kanten gewichtet oder ungewichtet, gerichtet oder ungerichtet sind.
 - (b) Beschreiben Sie einen Algorithmus, um Ihren Graphen aus der vorherigen Frage in (a) zu konstruieren. Sie können davon ausgehen, dass Ihre Songs in einer Datenstruktur gespeichert sind, die diesen Schritt am einfachsten macht. Nehmen Sie an, Sie haben eine Methode `makeEdge(v1, v2, w)`, die eine Kante von `v1` nach `v2` mit dem Gewicht `w` erzeugt.
 - (c) Beschreiben Sie einen Algorithmus, den Sie auf dem Graphen, den Sie gerade erstellt haben, ausführen könnten, um die Liste der Songs zu finden, die Sie spielen können, um am schnellsten zu “Wildest Dreams” zu gelangen, ohne das Publikum zu enttäuschen.
 - (d) Wie lange dauert es, bis Sie die Liste der Lieder gefunden haben? Geben Sie die Zeit an, die Sie benötigen, um Ihren Graphen zu erstellen und die Liste der Lieder zu finden. Geben Sie eine vereinfachte O-Notation in Bezug auf die von Ihnen benötigten Variablen an.

9. Kürzeste Pfade in einem gewichteten Graphen.



- (a) Bestimmen Sie den kürzesten Pfad vom Startknoten A aus zu allen anderen Knoten des obigen Graphen. Sie können dazu folgende Tabelle nutzen. Die kürzesten Abstände vom Knoten A aus sollen am Ende in der Spalte Distanz stehen.

Knoten	Distanz	Vorgänger	Verarbeitet?
A			
B			
C			
D			
E			
F			
G			

- (b) Zeichnen Sie den Baum kürzester Pfade von Startknoten A aus.

10. Ein soziales Netzwerk möchte Empfehlungen für neue Freunde basierend auf gemeinsamen Freunden geben. Entwickeln Sie einen Algorithmus in Pseudocode oder Wörtern, der einem Nutzer andere Nutzer zur Freundschaft vorschlägt. Dabei sollen solche Nutzer vorgeschlagen werden, die mit möglichst vielen Freunden des ursprünglichen Nutzers befreundet sind. In welcher Datenstruktur würden Sie die Anzahl gemeinsamer Freunde pro durchsuchten Nutzer speichern? Folgender Graph dient als Hilfestellung.

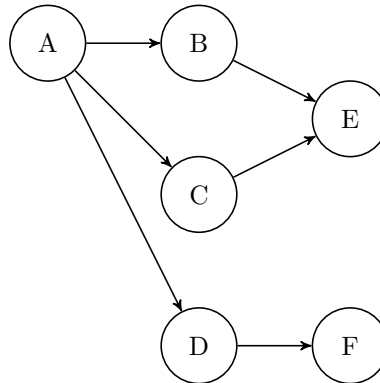


Abbildung 6: Der Knoten A ist der Nutzer dem neue Freunde vorgeschlagen werden sollen. Knoten B, C und D sind bereits Freunde von Knoten A. Der Algorithmus sollte Knoten E dem Nutzer A als neuen Freund vorschlagen, da E mit zwei Freunden von Knoten A befreundet ist. Knoten F soll Knoten A erst vorgeschlagen werden, wenn Knoten E bereits Knoten A vorgeschlagen wurde.

11. Erweitern Sie den iterativen Algorithmus der Tiefensuche (DFS) so, dass er das Erreichbarkeitsproblem löst. Das Erreichbarkeitsproblem lautet: Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t? Ihr Algorithmus soll also True zurückliefern, wenn es einen Pfad gibt, sonst soll der Algorithmus False zurückliefern.

Der iterative Algorithmus der Tiefensuche lautet:

Algorithm 1 Iterative Tiefensuche (DFS)

```

1: function dfs(Graph graph, Vertex start) {
2:   Stack<Vertex> perimeter = new Stack<>();
3:   Set<Vertex> visited = new Set<>();
4:
5:   perimeter.add(start);
6:
7:   while (!perimeter.isEmpty()) {
8:     Vertex from = perimeter.remove();
9:     if (!visited.contains(from)) {
10:      for (Edge edge : graph.edgesFrom(from)) {
11:        Vertex to = edge.to();
12:        if (!visited.contains(to))
13:          perimeter.add(to);
14:      }
15:      visited.add(from);
16:    }
17:  }
18: }
```

Aufgabe: Geben Sie mit einer kurzen Begründung an, zwischen welchen Zeilen des Codes neuer Code hinzugefügt oder entfernt werden muss, um das Erreichbarkeitsproblem zu lösen. Code, den Sie hinzufügen kann Pseudocode sein. Es muss kein korrekter Java oder Python Code sein. Sie müssen nur die Zeilen angeben bei denen etwas geändert werden muss. Sie müssen nicht den vollständigen Code des gesamten Algorithmus abschreiben.

12. Modellieren Sie ein Rezept zum Kuchen backen als Graph und geben Sie die Mindestdauer für den Gesamtvorgang an unter der Annahme, dass parallel durchführbare Vorgänge die Dauer der einzelnen Vorgänge nicht verlängert. Das Rezept hat die folgenden 9 Schritte und die Abhängigkeiten sind wie folgt gegeben.

1. Zutaten abwiegen: Muss als erstes gemacht werden, keine Abhängigkeiten. Dauer: 5 Minuten.
2. Zutaten mischen: Abhängig von "Zutaten abwiegen". Dauer: 4 Minuten
3. Teig kneten: Abhängig von "Zutaten mischen". Dauer: 10 Minuten
4. Backform vorbereiten: Kann unabhängig von den ersten Schritten durchgeführt werden. Dauer: 1 Minute
5. Teig in die Backform füllen: Abhängig von "Teig kneten" und "Backform vorbereiten". Dauer: 1 Minute
6. Ofen vorheizen: Kann parallel zu anderen Schritten erfolgen, aber muss vor dem Backen abgeschlossen sein. Dauer: 22 Minuten
7. Kuchen backen: Abhängig von "Teig in die Backform füllen" und "Ofen vorheizen". Dauer: 20 Minuten
8. Kuchen abkühlen lassen: Abhängig von "Kuchen backen". Dauer: 30 Minuten.
9. Kuchen verzieren: Abhängig von "Kuchen abkühlen lassen". Dauer: 8 Minuten.

(a) Graph:

(b) Gesamtdauer des Vorgangs: